



Deep Learning

Tutorial 11: Semantic Segmentation using Pre-Trained Model

(DeepLabV3+)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

- Test some model on the developed data
- Implement the Mask RCNN

Implementation of these tasks are in PyTorch:

Semantic Segmentation using Pre-Trained Model(DeepLabV3+)

```
import torch

from torchvision import models, transforms

from PIL import Image

import numpy as np

import matplotlib.pyplot as plt

import cv2

model = models.segmentation.deeplabv3_resnet101(pretrained=True)
model.eval()

image_path = "111.jpeg"
image = Image.open(image_path).convert("RGB")

preprocess = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) ]) input_tensor =
preprocess(image).unsqueeze(0)
with torch.no_grad():

    output = model(input_tensor)['out'][0]

output_predictions = output.argmax(0)
def decode_segmentation(mask, num_classes=21):
    colors = np.array([
        [0, 0, 0], [128, 0, 0], [0, 128, 0], [128, 128, 0],
        [0, 0, 128], [128, 0, 128], [0, 128, 128], [128, 128, 128],
        [64, 0, 0], [192, 0, 0], [64, 128, 0], [192, 128, 0],
        [64, 0, 128], [192, 0, 128], [64, 128, 128], [192, 128, 128],
        [0, 64, 0], [128, 64, 0], [0, 192, 0], [128, 192, 0],
        [0, 64, 128]
    ])

    r = np.zeros_like(mask, dtype=np.uint8) g
    = np.zeros_like(mask, dtype=np.uint8) b =
    np.zeros_like(mask, dtype=np.uint8)
```

```

for cls in range(num_classes):
    idx = mask == cls
    r[idx] = colors[cls, 0]
    g[idx] = colors[cls, 1]
    b[idx] = colors[cls, 2]

return np.stack([r, g, b], axis=2)

```

```

segmentation_map = decode_segmentation(output_predictions.numpy()) plt.figure(figsize=(10,
5))

```

```

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(image)

```

```

plt.subplot(1, 2, 2)
plt.title("Segmentation")
plt.imshow(segmentation_map)

```

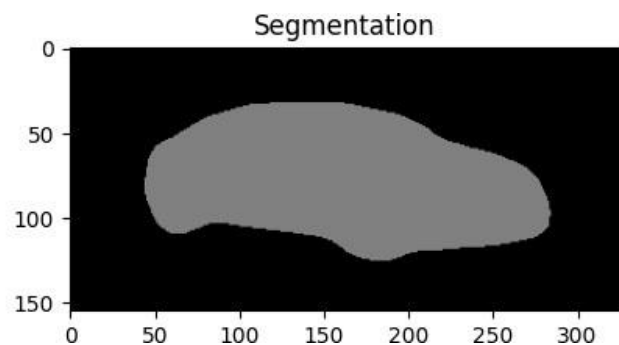
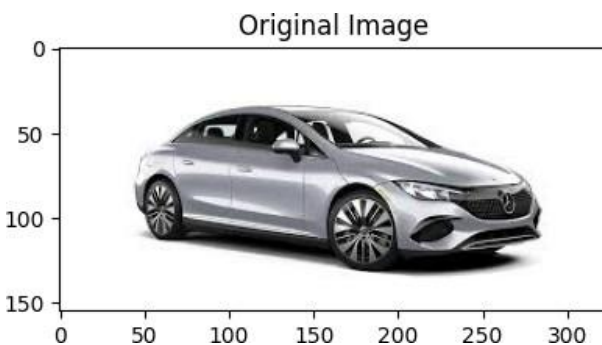
```

plt.show()

```

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead. warnings.warn(

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or 'None' for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing 'weights=DeepLabV3_ResNet101_Weights.COCO_WITH_VOC_LABELS_V1'. You can also use 'weights=DeepLabV3_ResNet101_Weights.DEFAULT' to get the most up-to-date weights. warnings.warn(msg)



```

import torch

```

```

from torchvision import models, transforms

```

```

from PIL import Image

```

```

import numpy as np

```

```

import matplotlib.pyplot as plt

```

```

import cv2

```

```

model = models.segmentation.deeplabv3_resnet101(pretrained=True)
model.eval()

```

```

image_path = "222.jpeg"
image = Image.open(image_path).convert('RGB')

```

```

preprocess = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) ]) input_tensor =
preprocess(image).unsqueeze(0)
with torch.no_grad():

    output = model(input_tensor)['out'][0]

output_predictions = output.argmax(0)
def decode_segmentation(mask, num_classes=21):
    colors = np.array([
        [0, 0, 0], [128, 0, 0], [0, 128, 0], [128, 128, 0],
        [0, 0, 128], [128, 0, 128], [0, 128, 128], [128, 128, 128],
        [64, 0, 0], [192, 0, 0], [64, 128, 0], [192, 128, 0],
        [64, 0, 128], [192, 0, 128], [64, 128, 128], [192, 128, 128],
        [0, 64, 0], [128, 64, 0], [0, 192, 0], [128, 192, 0],
        [0, 64, 128]
    ])

    r = np.zeros_like(mask, dtype=np.uint8) g
    = np.zeros_like(mask, dtype=np.uint8) b =
    np.zeros_like(mask, dtype=np.uint8)

    for cls in range(num_classes):

        idx = mask == cls
        r[idx] = colors[cls, 0]
        g[idx] = colors[cls, 1]
        b[idx] = colors[cls, 2]

    return np.stack([r, g, b], axis=2)

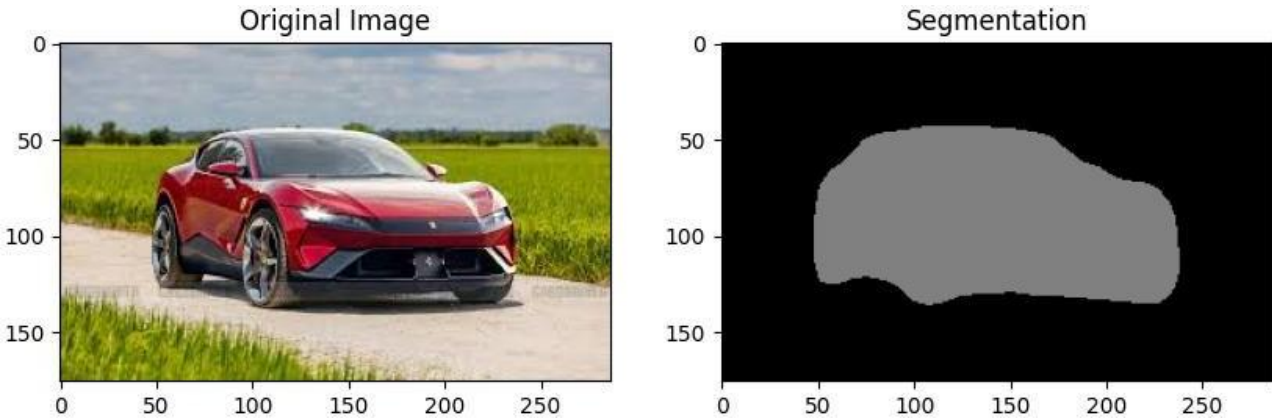
segmentation_map = decode_segmentation(output_predictions.numpy()) plt.figure(figsize=(10,
5))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(image)

plt.subplot(1, 2, 2)
plt.title("Segmentation")
plt.imshow(segmentation_map)

plt.show()

```



Tasks:

- Test some model on the developed data
- **Implement the Mask RCNN**

Step 0 — Install dependencies

!pip install torch torchvision

!pip install opencv-python

!pip install matplotlib

!pip install pillow

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cu126) Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.0+cu126) Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.0) Requirement already satisfied: typing-extensions<=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)

Requirement already satisfied: sympy<=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)

Requirement already satisfied: networkx<=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6) Requirement

already satisfied: fsspec<=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)

Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.77)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch) (3.3.20)

Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)

Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)

Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)

Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)

Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)

Step # 1 :Unzip TumorSegmentation.zip file which is uploaded to colab session storage,
 Converted COCO polygons to masks, at last get all image and mask paths recursively

```
import zipfile
```

```
import os
```

```
zip_path = "/content/TumorSegmentation.zip"
```

```

extract_path = "/content/TumorSegmentation"

with zipfile.ZipFile(zip_path, 'r') as z:
    z.extractall(extract_path)

print("Dataset extracted to:", extract_path)
import json

json_path = None

for root, dirs, files in os.walk(extract_path):
    for f in files:
        if f.endswith(".json"):
            json_path = os.path.join(root, f)
            break

    if json_path:
        break

if json_path is None:
    raise FileNotFoundError("No JSON annotation found!")
else:
    print("Using annotation file:", json_path)

with open(json_path, "r") as f:
    coco = json.load(f)

images = coco["images"]
annotations = coco["annotations"]
image_dict = {img["id"]: img["file_name"] for img in images}
import cv2

import numpy as np

mask_dir = os.path.join(extract_path, "masks")
os.makedirs(mask_dir, exist_ok=True)

for ann in annotations:
    img_id = ann["image_id"]
    img_file = image_dict[img_id]

    # Find image path recursively

    img_path = None

    for root, dirs, files in os.walk(extract_path):
        if img_file in files:
            img_path = os.path.join(root, img_file)
            break
    if img_path is None:

```

continue

```
img = cv2.imread(img_path)
if img is None:
```

continue

```
h, w = img.shape[:2]
```

```
mask = np.zeros((h, w), dtype=np.uint8) seg =
```

```
ann["segmentation"]
```

```
for polygon in seg:
```

```
    poly_pts = np.array(polygon).reshape(-1,2).astype(np.int32)
    cv2.fillPoly(mask, [poly_pts], 255)
```

```
mask_path = os.path.join(mask_dir, img_file.replace(".jpg", "_mask.png"))
cv2.imwrite(mask_path, mask)
```

```
print("Masks created in:", mask_dir)
```

```
from glob import glob
```

Recursively find images

```
image_paths = []
```

```
for root, dirs, files in os.walk(extract_path):
```

```
    for f in files:
```

```
        if f.lower().endswith((".jpg", ".jpeg", ".png")):
            image_paths.append(os.path.join(root, f))
```

Recursively find masks

```
mask_paths = []
```

```
for root, dirs, files in os.walk(mask_dir):
```

```
    for f in files:
```

```
        if f.endswith("_mask.png"):
            mask_paths.append(os.path.join(root, f))
```

Sort lists to match order

```
image_paths = sorted(image_paths)
```

```
mask_paths = sorted(mask_paths)
```

```
print("Total images found:", len(image_paths))
```

```
print("Total masks found:", len(mask_paths))
```

```
if len(image_paths)==0 or len(mask_paths)==0:
```

```
    raise ValueError("No images or masks found! Check ZIP contents.")
```

Dataset extracted to: /content/TumorSegmentation

Using annotation file: /content/TumorSegmentation/TumorSegmentation/_annotations.coco.json Masks

created in: /content/TumorSegmentation/masks

Total images found: 858

Total masks found: 429

Step # 2 : Test DeepLabv3 on dataset

```
import torch

from torchvision import models, transforms

from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

import random

# Load DeepLabv3 pre-trained model

model = models.segmentation.deeplabv3_resnet101(pretrained=True)
model.eval()

preprocess = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])

def decode_segmentation(mask, num_classes=21):
    colors = np.array([
        [0,0,0],[128,0,0],[0,128,0],[128,128,0],[0,0,128],[128,0,128],[0,128,128],[128,128,128],
        [64,0,0],[192,0,0],[64,128,0],[192,128,0],[64,0,128],[192,0,128],[64,128,128],[192,128,128],
        [0,64,0],[128,64,0],[0,192,0],[128,192,0],[0,64,128]
    ])

    r = np.zeros_like(mask, dtype=np.uint8)
    g = np.zeros_like(mask, dtype=np.uint8)
    b = np.zeros_like(mask, dtype=np.uint8)
    for cls in range(num_classes):
        idx = mask==cls
        r[idx]=colors[cls,0]
        g[idx]=colors[cls,1]
        b[idx]=colors[cls,2]
    return np.stack([r,g,b], axis=2)

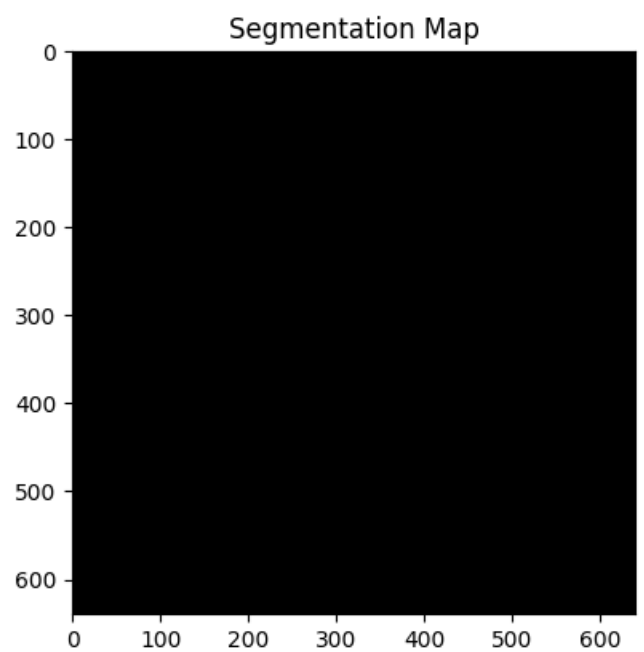
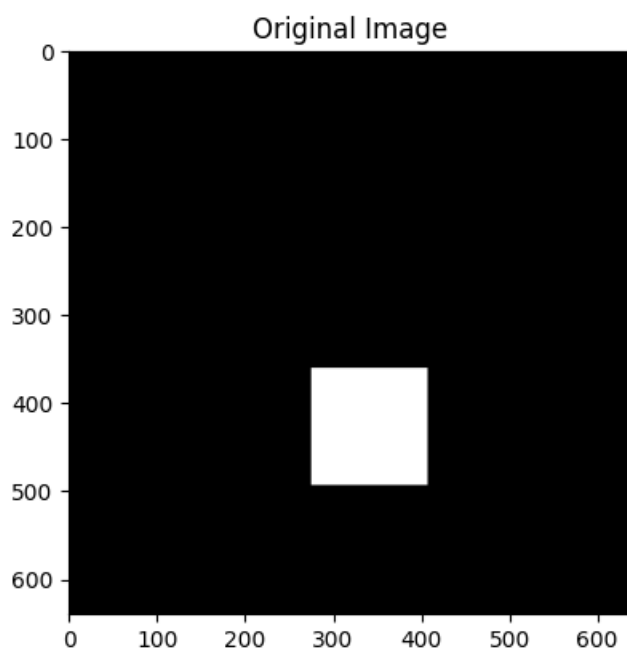
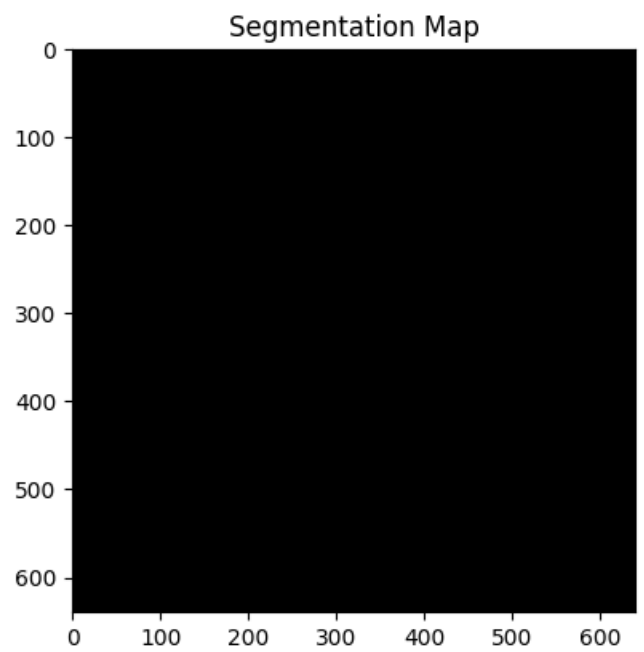
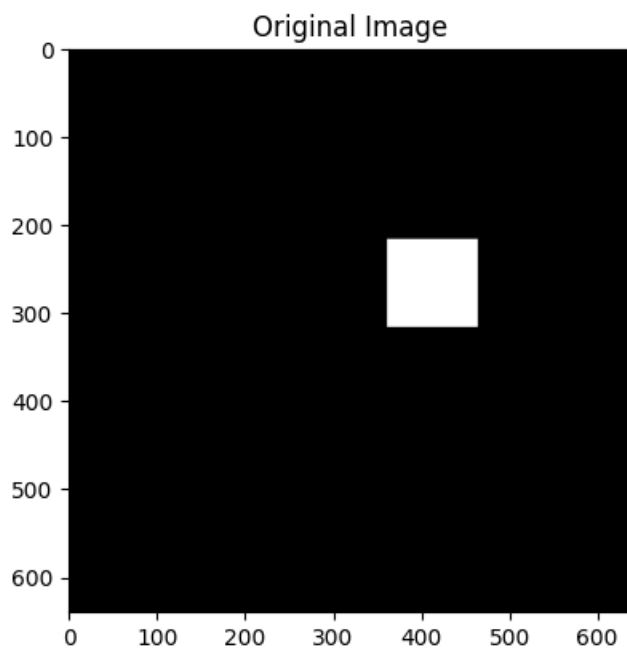
# Test on up to 3 images

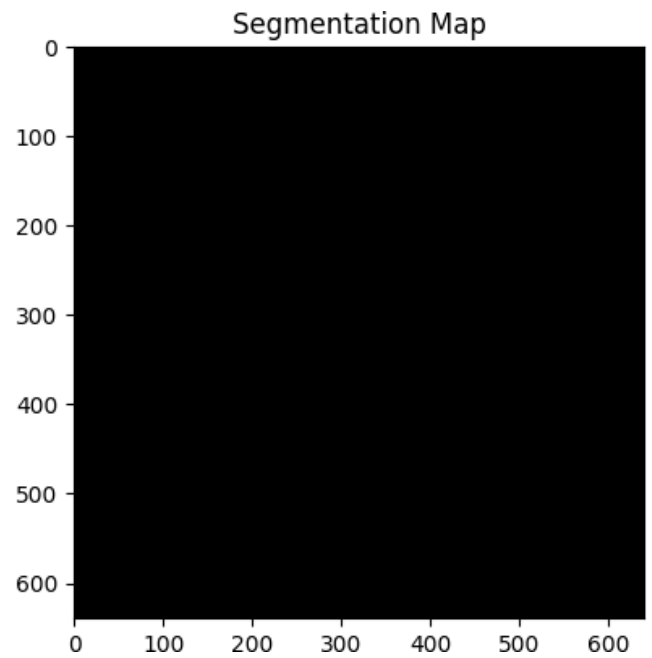
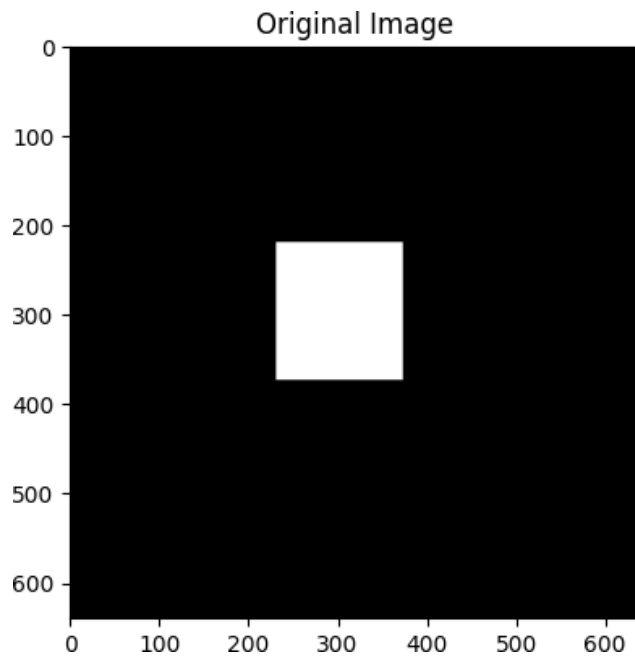
num_samples = min(3, len(image_paths))
sample_paths = random.sample(image_paths, num_samples)

for img_path in sample_paths:

    image = Image.open(img_path).convert("RGB")
    input_tensor = preprocess(image).unsqueeze(0)
    with torch.no_grad():
        output = model(input_tensor)['out'][0]
    output_predictions = output.argmax(0).numpy()
    segmentation_map = decode_segmentation(output_predictions)
```

```
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(image)
plt.subplot(1,2,2)
plt.title("Segmentation Map")
plt.imshow(segmentation_map)
plt.show()
```





Step #3 : Implement Mask R-CNN Dataset

```
import torch

from torch.utils.data import Dataset, DataLoader
import torchvision
import cv2

from PIL import Image
import numpy as np

class TumorDataset(Dataset):

    def __init__(self, image_paths, mask_paths, transforms=None):
        self.image_paths = image_paths
        self.mask_paths = mask_paths
        self.transforms = transforms

    def __len__(self):

        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        mask_path = self.mask_paths[idx]
```

```
img = Image.open(img_path).convert('RGB')
```

```

mask = cv2.imread(mask_path,0)
obj_ids = np.unique(mask)
obj_ids = obj_ids[1:] #remove background

if len(obj_ids) > 0:

    masks = mask==obj_ids[:,None,None]

else:
    masks = np.zeros((0, mask.shape[0], mask.shape[1]), dtype=np.uint8)
    boxes = []
    for m in masks:

        pos = np.where(m)
        xmin = np.min(pos[1])
        xmax = np.max(pos[1])
        ymin = np.min(pos[0])
        ymax = np.max(pos[0])
        boxes.append([xmin,ymin,xmax,ymax])
    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.ones((len(boxes),), dtype=torch.int64)
    masks = torch.as_tensor(masks, dtype=torch.uint8)
    target = {"boxes": boxes, "labels": labels, "masks": masks}

    return torchvision.transforms.ToTensor()(img), target

```

Step # 4:Load dataset and DataLoader

```

dataset = TumorDataset(image_paths, mask_paths)
data_loader = DataLoader(dataset, batch_size=2, shuffle=True, collate_fn=lambda x: tuple(zip(*x)))

```

Step # 05: Load pre-trained Mask R-CNN

```

from torchvision.models.detection import maskrcnn_resnet50_fpn

maskrcnn_model = maskrcnn_resnet50_fpn(pretrained=True)
maskrcnn_model.eval()
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=MaskRCNN_ResNet50_FPN_Weights.COCO_V1`. You can also use `weights=MaskRCNN_ResNet50_FPN_Weights.DEFAULT` to get the most up-to-date weights.
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/maskrcnn_resnet50_fpn_coco-bf2d0c1e.pth" to /root/.cache/torch/hub/checkpoints/maskrcnn_resnet50_fpn_coco-bf2d0c1e.pth
100% ██████████| 170M/170M [00:01<00:00, 151MB/s]
MaskRCNN(
  (transform): GeneralizedRCNNTransform(
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    Resize(min_size=(800,), max_size=1333, mode='bilinear')
  )
)

```

```

(backbone): BackboneWithFPN(
  (body): IntermediateLayerGetter(
    (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False) (bn1):
    FrozenBatchNorm2d(64, eps=0.0)
    (relu): ReLU(inplace=True)
    (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
    (layer1): Sequential(
      (0): Bottleneck(
        (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
        FrozenBatchNorm2d(64, eps=0.0)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
        FrozenBatchNorm2d(64, eps=0.0)

        (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
        FrozenBatchNorm2d(256, eps=0.0)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (1): FrozenBatchNorm2d(256, eps=0.0)
        )
      )

    )

    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
      FrozenBatchNorm2d(64, eps=0.0)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
      FrozenBatchNorm2d(64, eps=0.0)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
      FrozenBatchNorm2d(256, eps=0.0)
      (relu): ReLU(inplace=True)
    )

    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
      FrozenBatchNorm2d(64, eps=0.0)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
      FrozenBatchNorm2d(64, eps=0.0)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
      FrozenBatchNorm2d(256, eps=0.0)
      (relu): ReLU(inplace=True)
    )

  )

  (layer2): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): FrozenBatchNorm2d(128, eps=0.0)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False) (bn2):
      FrozenBatchNorm2d(128, eps=0.0)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): FrozenBatchNorm2d(512, eps=0.0)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): FrozenBatchNorm2d(512, eps=0.0)
      )
    )
  )

```

```

)

(1): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(128, eps=0.0)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(128, eps=0.0)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(512, eps=0.0)
  (relu): ReLU(inplace=True)
)

(2): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(128, eps=0.0)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(128, eps=0.0)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(512, eps=0.0)
  (relu): ReLU(inplace=True)
)

(3): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(128, eps=0.0)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(128, eps=0.0)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(512, eps=0.0)
  (relu): ReLU(inplace=True)
)

)

(layer3): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): FrozenBatchNorm2d(256, eps=0.0)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False) (bn2):
    FrozenBatchNorm2d(256, eps=0.0)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): FrozenBatchNorm2d(1024, eps=0.0)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): FrozenBatchNorm2d(1024, eps=0.0)
    )
  )

  (1): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): FrozenBatchNorm2d(256, eps=0.0)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
    FrozenBatchNorm2d(256, eps=0.0)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): FrozenBatchNorm2d(1024, eps=0.0)
    (relu): ReLU(inplace=True)
  )
)

```

```
(2): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(256, eps=0.0)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(256, eps=0.0)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(1024, eps=0.0)
  (relu): ReLU(inplace=True)
)
```

```
(3): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(256, eps=0.0)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(256, eps=0.0)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(1024, eps=0.0)
  (relu): ReLU(inplace=True)
)
```

```
(4): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(256, eps=0.0)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(256, eps=0.0)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(1024, eps=0.0)
  (relu): ReLU(inplace=True)
)
```

```
(5): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(256, eps=0.0)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(256, eps=0.0)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(1024, eps=0.0)
  (relu): ReLU(inplace=True)
)
```

```
)
```

```
(layer4): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): FrozenBatchNorm2d(512, eps=0.0)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False) (bn2):
    FrozenBatchNorm2d(512, eps=0.0)
    (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): FrozenBatchNorm2d(2048, eps=0.0)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): FrozenBatchNorm2d(2048, eps=0.0)
    )
  )
)
```

```

(1): Bottleneck(
  (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(512, eps=0.0)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(512, eps=0.0)
  (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(2048, eps=0.0)
  (relu): ReLU(inplace=True)
)

(2): Bottleneck(
  (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): FrozenBatchNorm2d(512, eps=0.0)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
  FrozenBatchNorm2d(512, eps=0.0)
  (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): FrozenBatchNorm2d(2048, eps=0.0)
  (relu): ReLU(inplace=True)
)

)

)

(fpn): FeaturePyramidNetwork(
  (inner_blocks): ModuleList(
    (0): Conv2dNormActivation(
      (0): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
    )

    (1): Conv2dNormActivation(
      (0): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1))
    )

    (2): Conv2dNormActivation(
      (0): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1))
    )

    (3): Conv2dNormActivation(
      (0): Conv2d(2048, 256, kernel_size=(1, 1), stride=(1, 1))
    )
  )

  (layer_blocks): ModuleList(
    (0-3): 4 x Conv2dNormActivation(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    )
  )

  (extra_blocks): LastLevelMaxPool()
)

)

```



```

(rpn): RegionProposalNetwork(
  (anchor_generator): AnchorGenerator()
  (head): RPNHead(
    (conv): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): ReLU(inplace=True)
      )
    )
  )

  (cls_logits): Conv2d(256, 3, kernel_size=(1, 1), stride=(1, 1))
  (bbox_pred): Conv2d(256, 12, kernel_size=(1, 1), stride=(1, 1))
)

)

(roi_heads): RoIHeads(
  (box_roi_pool): MultiScaleRoIAlign(featmap_names=['0', '1', '2', '3'], output_size=(7, 7), sampling_ratio=2)
  (box_head): TwoMLPHead(
    (fc6): Linear(in_features=12544, out_features=1024, bias=True) (fc7):
    Linear(in_features=1024, out_features=1024, bias=True)
  )

  (box_predictor): FastRCNNPredictor(
    (cls_score): Linear(in_features=1024, out_features=91, bias=True) (bbox_pred):
    Linear(in_features=1024, out_features=364, bias=True)
  )

  (mask_roi_pool): MultiScaleRoIAlign(featmap_names=['0', '1', '2', '3'], output_size=(14, 14), sampling_ratio=
2)
  (mask_head): MaskRCNNHeads(
    (0): Conv2dNormActivation(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU(inplace=True)
    )

    (1): Conv2dNormActivation(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU(inplace=True)
    )

    (2): Conv2dNormActivation(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU(inplace=True)
    )

    (3): Conv2dNormActivation(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU(inplace=True)
    )
  )

  (mask_predictor): MaskRCNNPredictor(
    (conv5_mask): ConvTranspose2d(256, 256, kernel_size=(2, 2), stride=(2, 2)) (relu):
    ReLU(inplace=True)
    (mask_fcn_logits): Conv2d(256, 91, kernel_size=(1, 1), stride=(1, 1))
  )
)

```

```
)  
)
```

Step #6 : Test Mask R-CNN on one image

```
import matplotlib.pyplot as plt
```

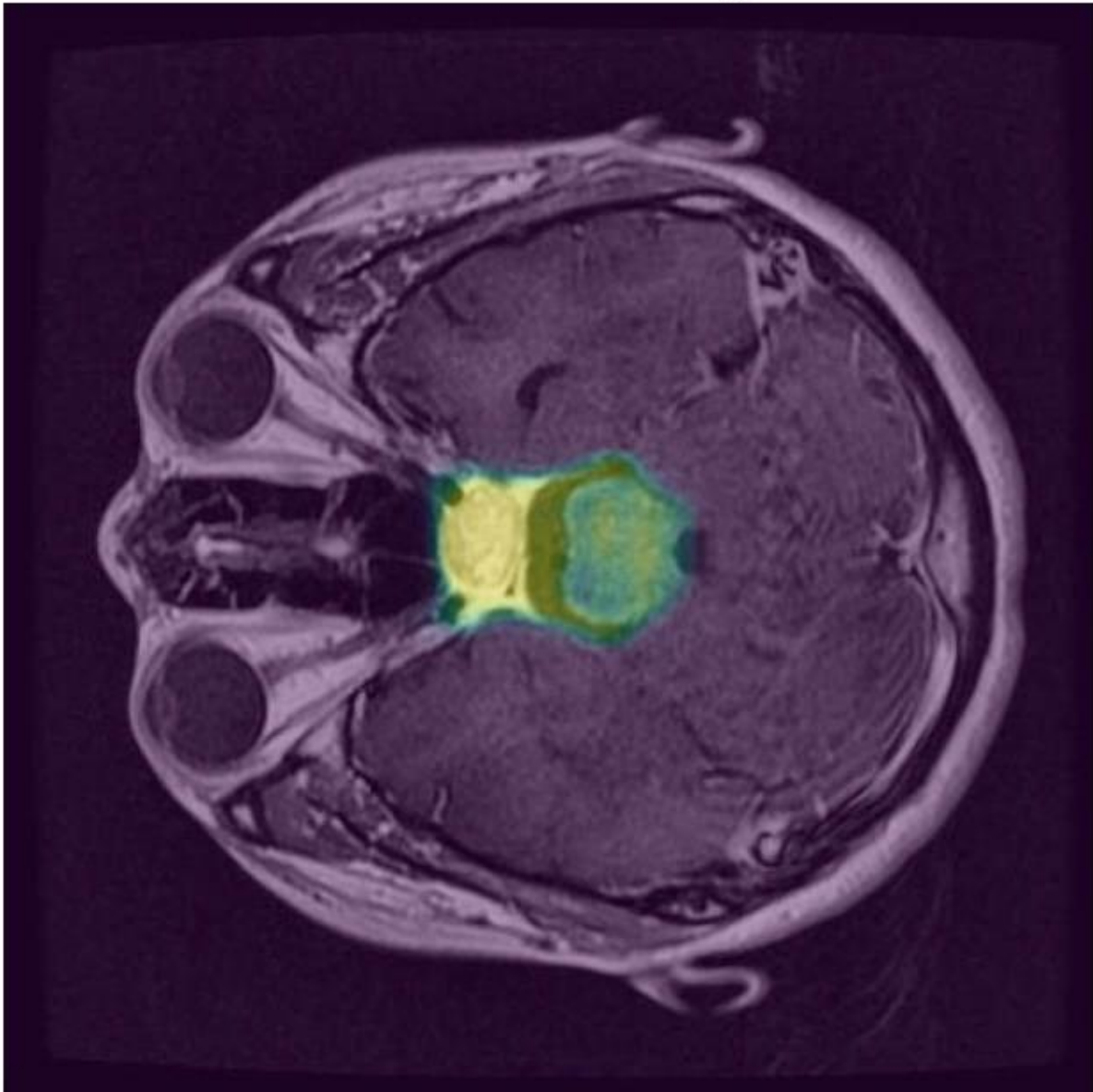
```
def visualize_maskrcnn_predictions(model, dataset, num_images=5): """  
    Runs Mask R-CNN on multiple images and visualizes predictions. """  
    model.eval()  
    total = min(num_images, len(dataset))  
  
    for i in range(total):  
  
        img, target = dataset[i]  
  
        with torch.no_grad():  
  
            prediction = model([img])  
  
            pred_masks = prediction[0]['masks'].cpu().numpy()          # (N,1,H,W)  
  
            pred_scores = prediction[0]['scores'].cpu().numpy()        # confidence scores  
  
            # Filter out weak masks (optional)  
            keep = pred_scores > 0.5  
            pred_masks = pred_masks[keep]  
  
            # Convert image back to numpy  
  
            img_np = img.permute(1, 2, 0).cpu().numpy()  
            img_np = (img_np - img_np.min()) / (img_np.max() + 1e-8) # normalize for display  
  
            plt.figure(figsize=(8, 8))  
            plt.imshow(img_np)  
            plt.title(f'Mask R-CNN Predictions: Image {i+1}')  
  
            # Plot masks with transparency  
  
            for m in pred_masks:  
  
                m = m[0] # remove extra dimension  
  
                plt.imshow(m, alpha=0.4)  
  
            plt.axis("off")
```

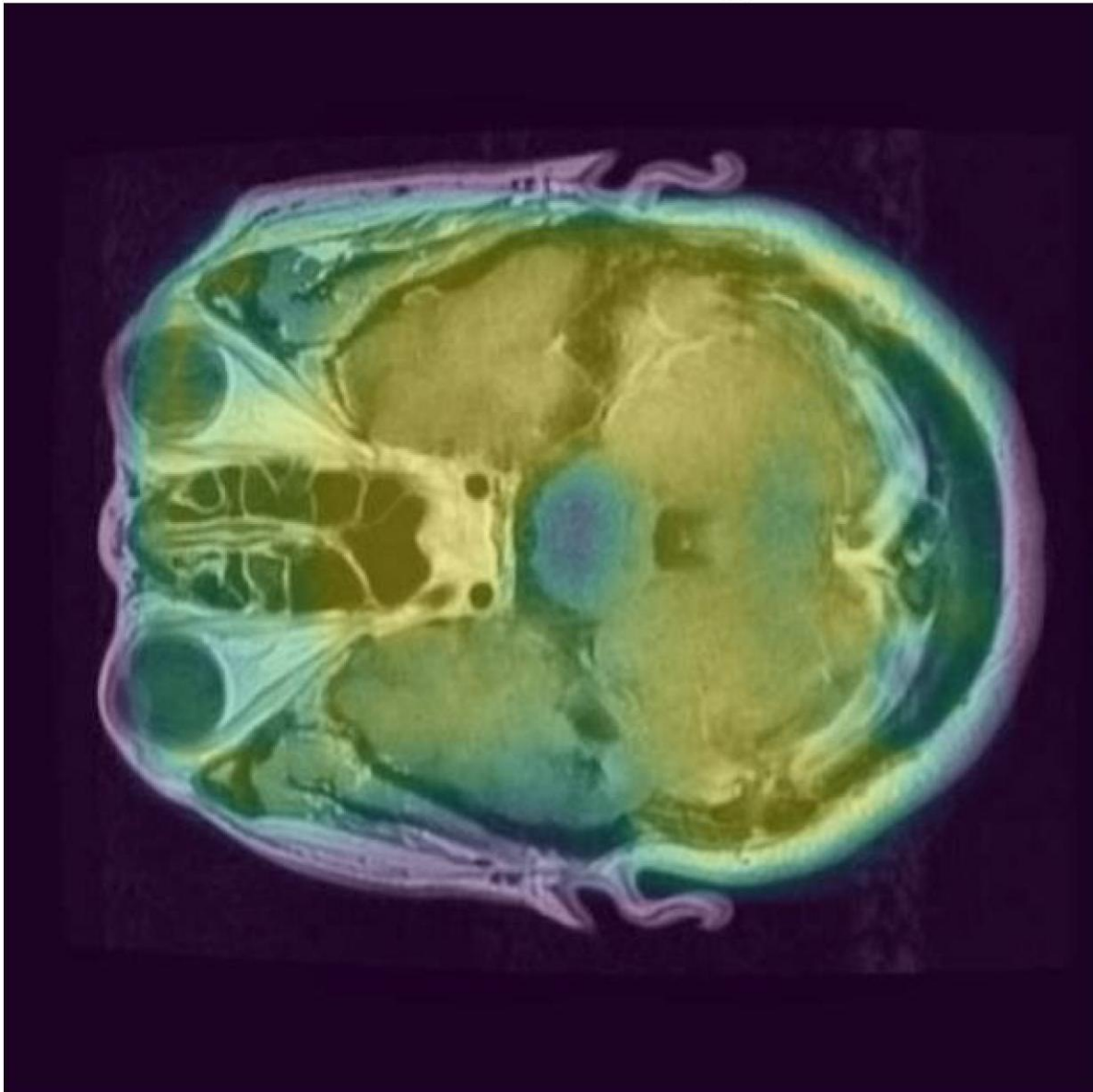
```
plt.show()
```

```
# Run on 5 images
```

```
visualize_maskrcnn_predictions(maskrcnn_model, dataset, num_images=5)
```

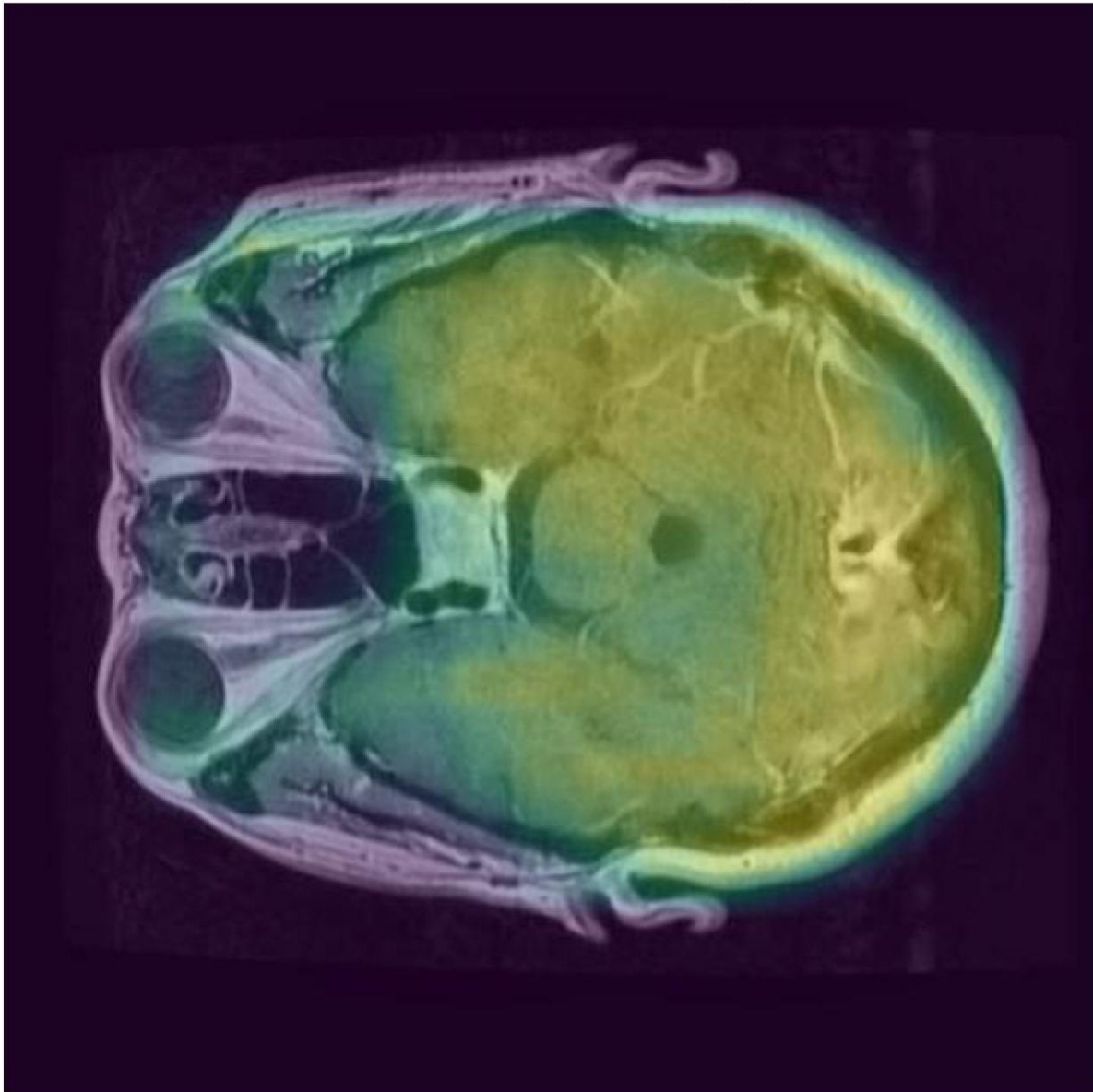
Mask R-CNN Predictions: Image 1



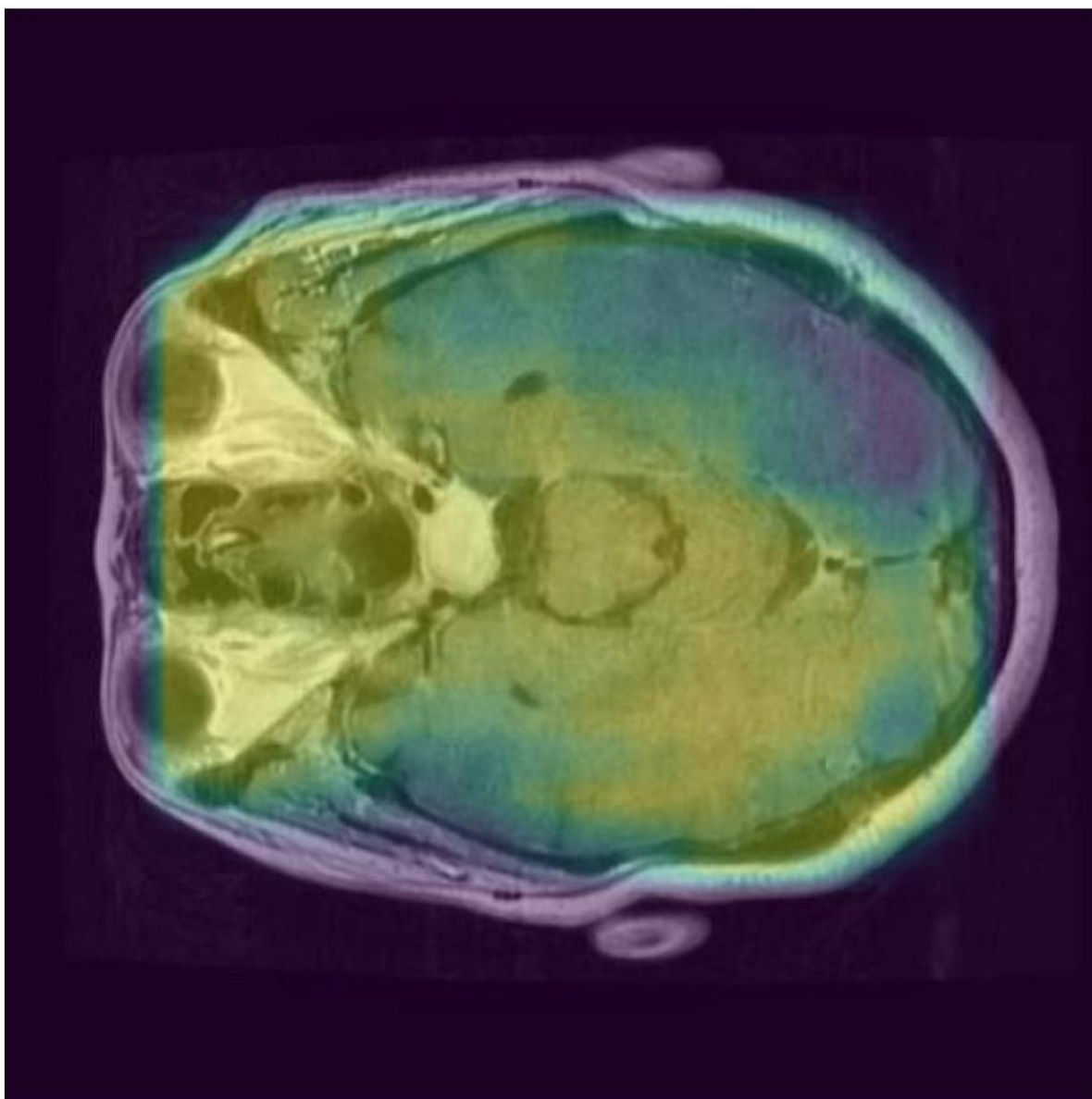


Mask R-CNiNPredictions: Image 2

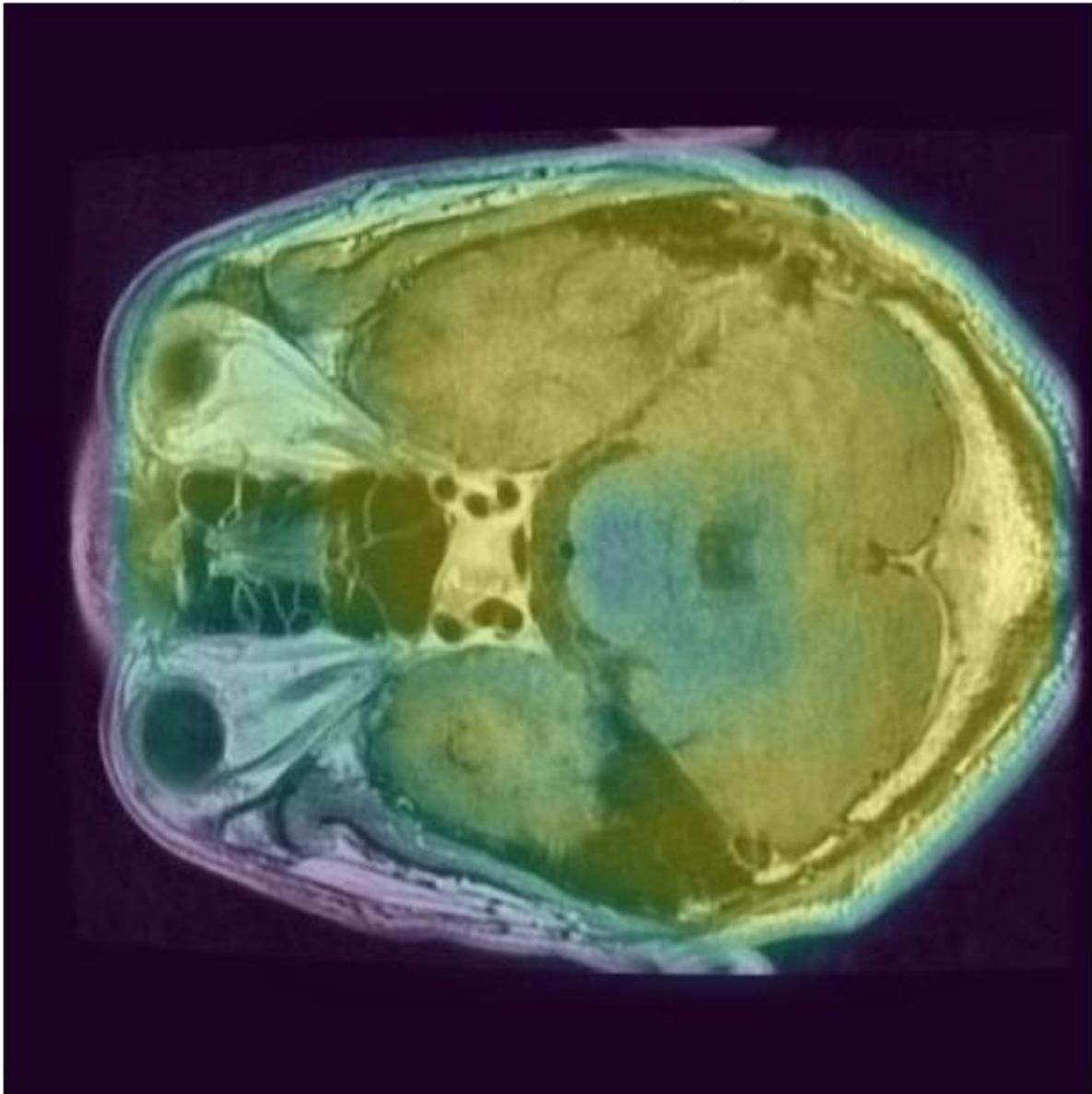
Mask R-CNiNPredictions: Image 3



Mask R-CNN Predictions: Image 4



Mask R-CNN Predictions: Image 5



Visualizing Original Image + True Mask

```
import matplotlib.pyplot as plt
import cv2

def plot_image_and_mask(img_path, mask_path):
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    mask = cv2.imread(mask_path, 0) # grayscale mask

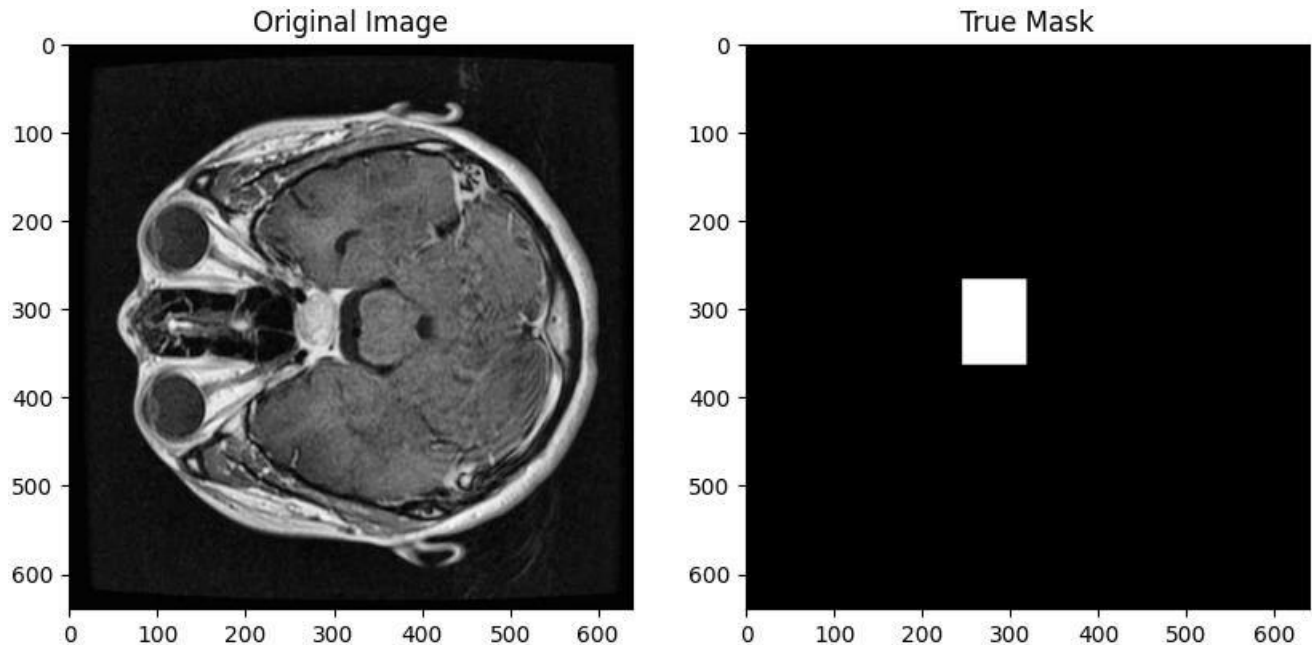
    plt.figure(figsize=(10,5))
    plt.subplot(1,2,1)
```

```
plt.title("Original Image")
plt.imshow(img)

plt.subplot(1,2,2)
plt.title("True Mask")
plt.imshow(mask, cmap="gray")
plt.show()
```

Example

```
plot_image_and_mask(image_paths[0], mask_paths[0])
```



Visualizing DeepLabv3 Predictions

```
from PIL import Image
```

```
import torch
```

```
from torchvision import transforms
```

```
import numpy as np
```

```
preprocess = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])
```

```
def decode_segmentation(mask, num_classes=21):
    colors = np.array([
        [0,0,0],[128,0,0],[0,128,0],[128,128,0],[0,0,128],[128,0,128],[0,128,128],[128,128,128],
        [64,0,0],[192,0,0],[64,128,0],[192,128,0],[64,0,128],[192,0,128],[64,128,128],[192,128,128],
        [0,64,0],[128,64,0],[0,192,0],[128,192,0],[0,64,128]
    ])
    return mask
```



```

r = np.zeros_like(mask, dtype=np.uint8) g
= np.zeros_like(mask, dtype=np.uint8) b =
np.zeros_like(mask, dtype=np.uint8)

```

```

for cls in range(num_classes):

```

```

    idx = mask==cls

```

```

    r[idx]=colors[cls,0]

```

```

    g[idx]=colors[cls,1]

```

```

    b[idx]=colors[cls,2]

```

```

return np.stack([r,g,b], axis=2)

```

```

def plot_deeplabv3_prediction(img_path):

```

```

    image = Image.open(img_path).convert("RGB")

```

```

    input_tensor = preprocess(image).unsqueeze(0)

```

```

    with torch.no_grad():

```

```

        output = model(input_tensor)['out'][0] pred_mask

```

```

    = output.argmax(0).numpy() seg_map =

```

```

    decode_segmentation(pred_mask)

```

```

plt.figure(figsize=(10,5))

```

```

plt.subplot(1,2,1) plt.title("Original
Image") plt.imshow(image)

```

```

plt.subplot(1,2,2) plt.title("Predicted
Segmentation")

```

```

plt.imshow(seg_map)

```

```

plt.show()

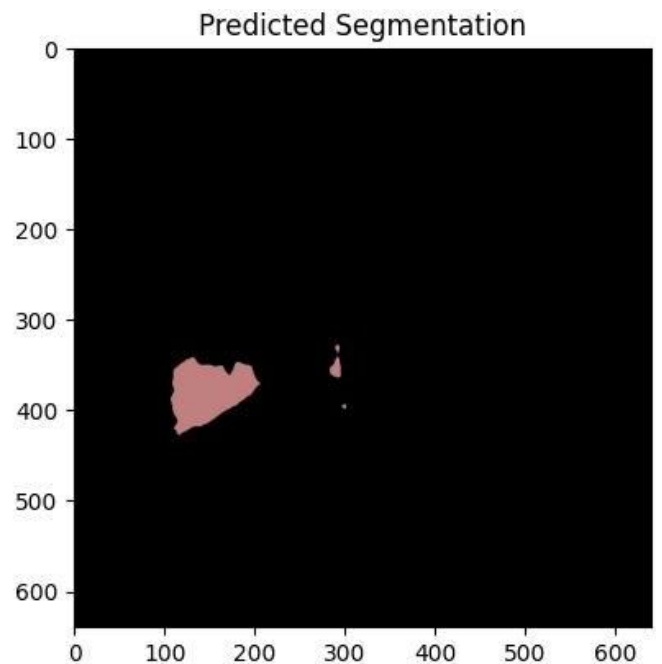
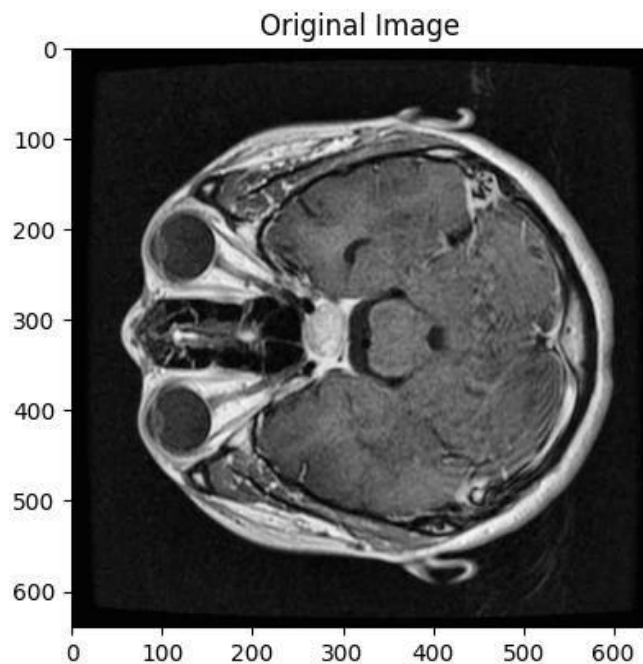
```

Example

```

plot_deeplabv3_prediction(image_paths[0])

```



Overlaying Predicted Mask on Original Image

```
def overlay_mask(img_path, mask, alpha=0.5):
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    mask_colored = cv2.applyColorMap(mask, cv2.COLORMAP_JET)
    overlay = cv2.addWeighted(img, 1-alpha, mask_colored, alpha, 0)

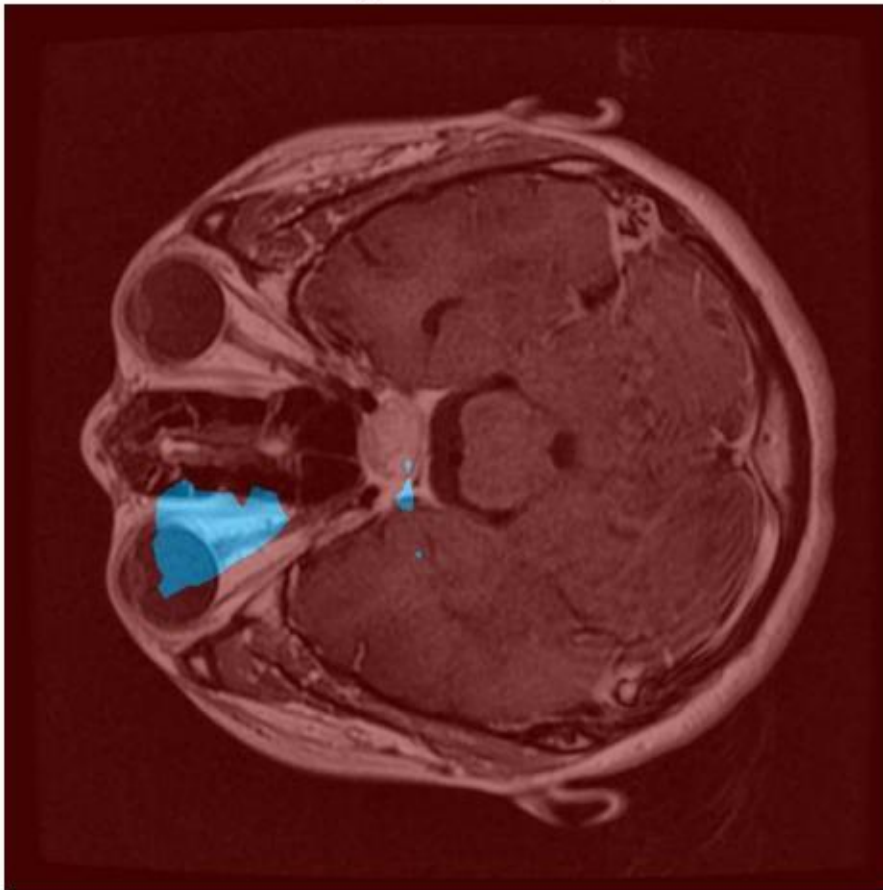
    plt.figure(figsize=(6,6))
    plt.title("Overlay Mask on Image")
    plt.imshow(overlay) plt.axis("off")
    plt.show()

# For DeepLabv3 prediction

image = Image.open(image_paths[0]).convert("RGB")
input_tensor = preprocess(image).unsqueeze(0) with torch.no_grad():
    output = model(input_tensor)['out'][0]
pred_mask = output.argmax(0).numpy().astype(np.uint8)*12 # scale 0-255 for visualization

overlay_mask(image_paths[0], pred_mask)
```

Overlay Mask on Image



Visualizing Mask R-CNN Predictions

```
sample_img, sample_target = dataset[0]
with torch.no_grad():

    prediction = maskrcnn_model([sample_img])

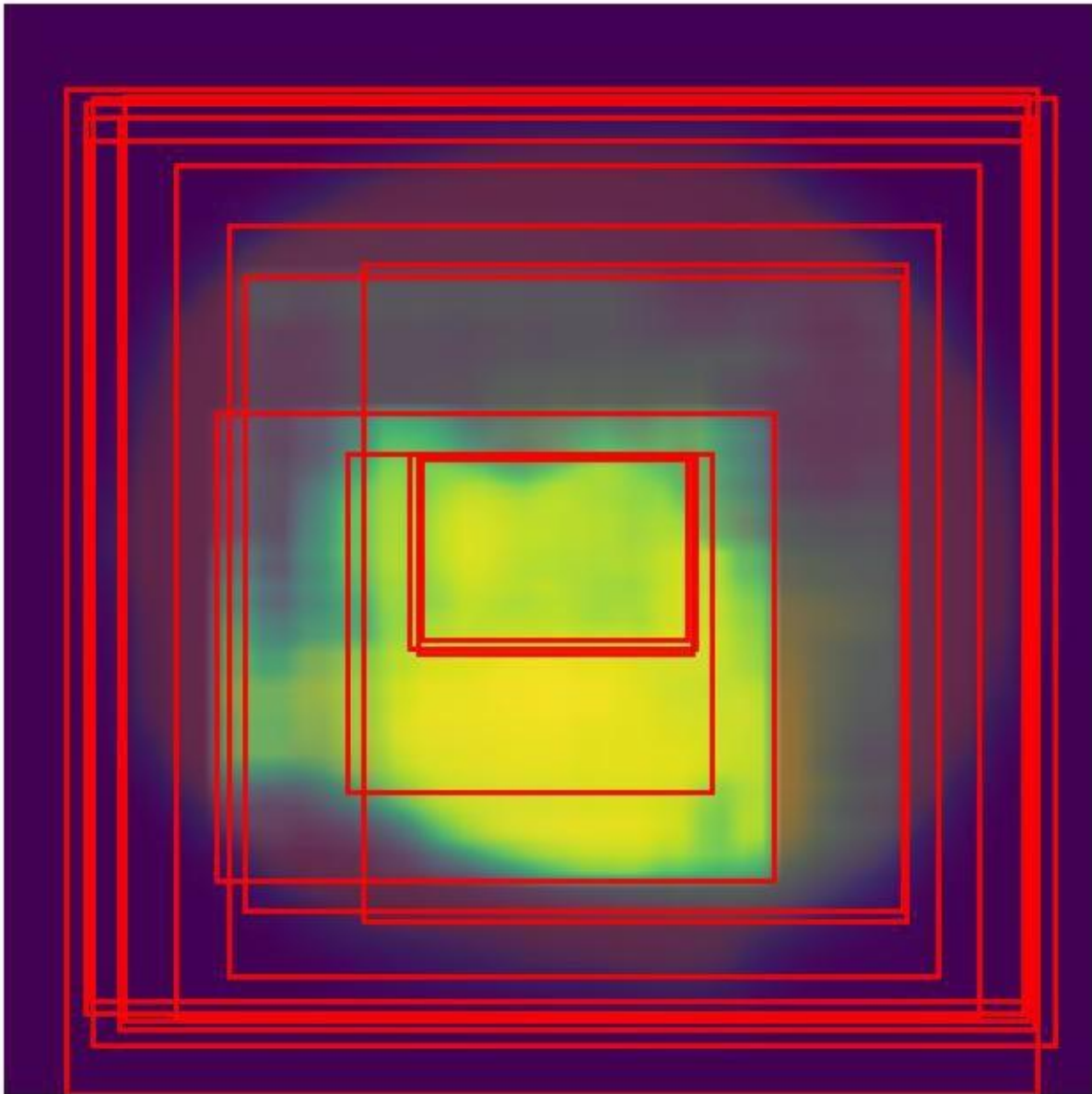
pred_masks = prediction[0]['masks'].cpu().numpy() pred_boxes =
prediction[0]['boxes'].cpu().numpy()

plt.figure(figsize=(8,8))
plt.imshow(sample_img.permute(1,2,0).numpy()) for
mask in pred_masks:
    mask = mask[0]
    plt.imshow(mask, alpha=0.5) # overlay each mask

for box in pred_boxes:

    xmin, ymin, xmax, ymax = box plt.gca().add_patch(plt.Rectangle((xmin,ymin),
    xmax-xmin, ymax-ymin,
                        fill=False, edgecolor='red', linewidth=2))
plt.title("Mask R-CNN Predictions")
plt.axis("off")
plt.show()
```

Mask R-CNN Predictions



Comparing True Mask vs Predicted Mask

```
def compare_true_pred(img_path, mask_path, pred_mask): img
    = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    true_mask = cv2.imread(mask_path,0)

    plt.figure(figsize=(15,5))
    plt.subplot(1,3,1)
    plt.title("Original Image")
    plt.imshow(img)
```

```

plt.subplot(1,3,2)
plt.title("True Mask")
plt.imshow(true_mask, cmap="gray")

plt.subplot(1,3,3)
plt.title("Predicted Mask Overlay")
overlay = cv2.addWeighted(img, 0.7, cv2.applyColorMap(pred_mask, cv2.COLORMAP_JET), 0.3, 0)
plt.imshow(overlay)
plt.show()

# Example for DeepLabv3

image =
Image.open(image_paths[0]).convert("RGB")
input_tensor =
preprocess(image).unsqueeze(0) with
torch.no_grad():
    output = model(input_tensor)['out'][0]
pred_mask = output.argmax(0).numpy().astype(np.uint8)*12

compare_true_pred(image_paths[0], mask_paths[0],
pred_mask)

```

